

RESOLUCIÓN DE MODELOS DE PROGRAMACIÓN LINEAL MEDIANTE MÉTODOS GRÁFICO, SIMPLEX Y DE LA GRAN M (Agosto de 2026)

Martin Pineda Jaramillo

Facultad de Ciencias Básicas e Ingeniería, Universidad de los Llanos, Villavicencio, Colombia

m.pineda@unillanos.edu.co, 160005227

Resumen

Este documento presenta el diseño y desarrollo de una aplicación interactiva construida en Python para la resolución de modelos de programación lineal. La herramienta consolida las soluciones mediante el método gráfico para problemas de dos variables, el método Simplex algorítmico para modelos estándar, y la extensión del Método de la Gran M para problemas de minimización que incluyen restricciones de límite inferior. El diseño del sistema se ha estructurado mediante Historias de Usuario, y abstraído a nivel general a través de diagramas UML de clases y secuencias. El despliegue unificado permite a los estudiantes de la asignatura de Optimización visualizar las regiones factibles, comprender iterativamente el cálculo de pivotes en los poliedros n-dimensionales y evaluar el impacto de las variables artificiales en la optimización.

Palabras clave: Método de la Gran M, Método Gráfico, Método Simplex, Optimización, Programación Lineal, Python.

I. INTRODUCCIÓN

La Programación Lineal es una de las herramientas más utilizadas dentro de la investigación de operaciones, ya que permite modelar y resolver problemas de asignación óptima de recursos limitados, sujetos a un conjunto de restricciones lineales. Su aplicación abarca distintas áreas como la producción industrial, la logística, la economía y la administración de proyectos, en donde se busca maximizar beneficios o minimizar costos.

Resolver manualmente estos problemas, especialmente cuando el número de variables y restricciones crece o cuando se incluyen operadores lógicos mixtos, resulta un proceso tedioso y propenso a errores de cálculo. Por esta razón, resulta de gran utilidad contar con herramientas informáticas capaces de automatizar dicho proceso, tanto con fines académicos como prácticos.

El presente trabajo tiene como objetivo desarrollar una aplicación de escritorio que permita resolver problemas de Programación Lineal utilizando los métodos más representativos: el Método Gráfico, aplicable únicamente a problemas de dos variables; el Método Simplex, aplicable a problemas de n variables con restricciones de menor o igual; y el Método de la Gran M, como extensión indispensable para abordar restricciones de mayor o igual, aplicado de forma exclusiva a problemas de minimización. Además, se documenta el proceso de análisis del software mediante historias de usuario, y su diseño mediante diagrama de clases y diagrama de secuencia, siguiendo buenas prácticas de ingeniería de software.

II. MARCO TEÓRICO

2.1 Programación Lineal:

Resolucion de problemas lineales

La Programación Lineal es una técnica matemática de optimización que consiste en maximizar o minimizar una función lineal, denominada función objetivo, sujeta a un conjunto de restricciones también lineales, expresadas como ecuaciones o inecuaciones. Todo problema de programación lineal está compuesto por tres elementos principales: la función objetivo, las variables de decisión y las restricciones, incluyendo la condición de no negatividad de las variables.

2.2 Método Gráfico:

El método gráfico es un procedimiento de solución aplicable exclusivamente a problemas con dos variables de decisión. Consiste en representar cada restricción como una recta en el plano cartesiano, determinar la región factible como la intersección de los semiplanos definidos por las restricciones, e identificar los vértices de dicha región. Posteriormente, se evalúa la función objetivo en cada uno de los vértices factibles, y el valor óptimo corresponde al vértice que maximiza o minimiza dicha función, según corresponda.

2.2.1 Ejemplo Matemático:

Para ilustrar el procedimiento, considere el siguiente problema clásico:

$$\text{Max } Z = 3x_1 + 5x_2$$

Sujeto a:

$$x_1 \leq 4$$

$$2x_2 \leq 12$$

$$3x_1 + 2x_2 \leq 18$$

$$x_1, x_2 \geq 0$$

Al graficar las tres restricciones junto con los ejes, se obtienen los siguientes vértices de la región factible: (0,0), (4,0), (4,3), (2,6) y (0,6). Evaluando Z en cada uno:

$$Z(0,0) = 0$$

$$Z(4,0) = 12$$

$$Z(4,3) = 27$$

$$Z(2,6) = 36$$

$$Z(0,6) = 30$$

El valor óptimo es $Z = 36$, alcanzado en el punto (2,6). Este es el mismo procedimiento que reproduce internamente la clase GraphicSolver.

2.3 Método Simplex:

El método Simplex, propuesto por George Dantzig, es un algoritmo iterativo utilizado para resolver problemas de programación lineal con cualquier número de variables. El método parte de una solución básica factible inicial e itera mejorando dicha solución mediante operaciones de pivoteo, hasta alcanzar la solución óptima.

2.4 Método de la Gran M:

Resolucion de problemas lineales

El Método Simplex tradicional asume que el origen es una solución básica factible inicial (típico en restricciones $\leq$). Cuando el modelo incluye restricciones de tipo mayor o igual ($\geq$), el origen deja de ser factible. El método de la Gran M soluciona esto introduciendo "variables artificiales" que permiten forzar una solución básica inicial geométrica. Para asegurar que estas variables artificiales no formen parte de la solución óptima final, se penalizan en la función objetivo asignándoles un coeficiente "M" de un valor excesivamente grande y positivo, dado que en este sistema el método se aplica exclusivamente para modelos de minimización.

2.5 Ingeniería de software:

Las historias de usuario son una técnica ágil utilizada para describir una funcionalidad que el sistema debe cumplir. El diagrama de clases representa la estructura, mientras que el diagrama de secuencia evidencia el orden en que se envían los mensajes entre los objetos a lo largo del tiempo.

III. DESARROLLO DE CONTENIDOS

3.1 Historias de usuario:

ID	Título	Descripción	Criterio de aceptación
HU-01	Configuración de variables y restricciones	Como usuario, quiero ingresar el número de variables y restricciones, para definir el tamaño de mi problema.	Al ingresar los valores y confirmar, el sistema genera los campos correspondientes.
HU-02	Selección del método de resolución	Como usuario, quiero elegir entre método Gráfico, Simplex o Gran M.	Al seleccionar un método y resolver, el sistema aplica la lógica correspondiente.
HU-03	Ingreso de la función objetivo y las restricciones	Como usuario, quiero ingresar la función objetivo y las restricciones con su operador ($\leq$, $\geq$) y valor.	El sistema arma correctamente el problema y detecta automáticamente si se requiere la Gran M.
HU-04	Selección del tipo de optimización	Como usuario, quiero elegir entre maximizar o minimizar.	El sistema aplica la configuración; si se selecciona Gran M, el sistema valida que la optimización sea estrictamente de minimización.
HU-05	Visualización de la solución obtenida	Como usuario, quiero ver el resultado del problema paso a paso.	El sistema muestra el valor óptimo, las iteraciones o la gráfica.

Tabla 1. Historias de usuario.

3.2 Arquitectura general del sistema:

El sistema sigue un enfoque orientado a objetos. Una clase base abstracta, LinearSolver, almacena los atributos comunes a cualquier problema de PL. De ella heredan GraphicSolver y SimplexSolver. Esta última fue extendida para integrar la lógica del método de la Gran M de manera dinámica en casos de minimización. La interfaz gráfica es gestionada por la clase LinearProgrammingApp.

Resolucion de problemas lineales

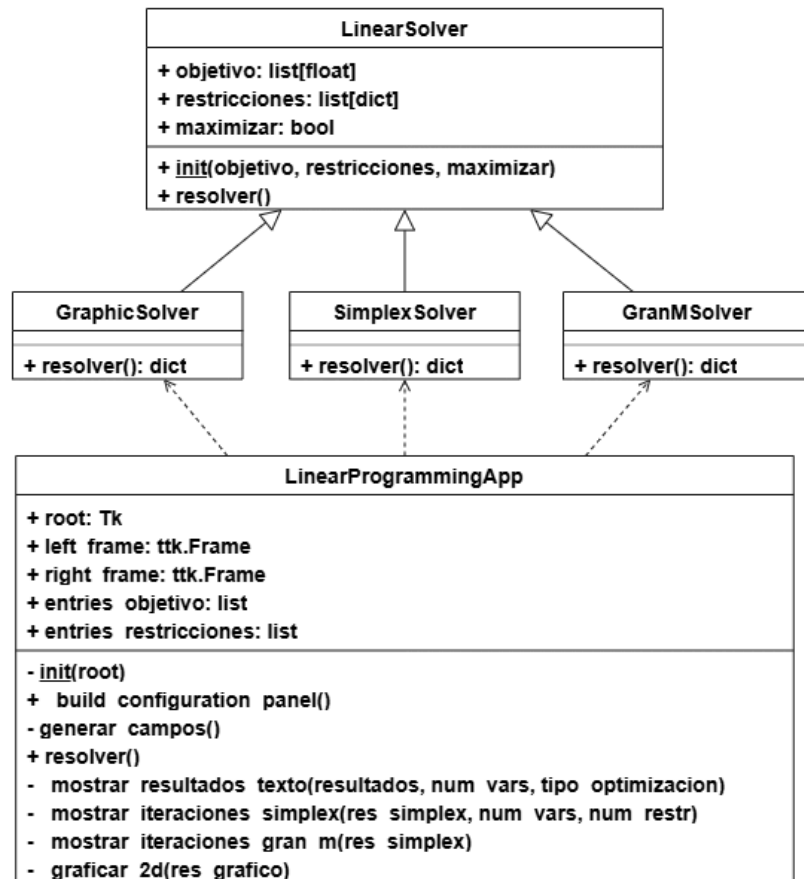


Figura 1. Diagrama de clases.

3.3 Diagrama de secuencia:

El diagrama documenta el flujo entre el usuario, la interfaz y el solver. Dependiendo de los operadores lógicos ingresados en las restricciones y si el objetivo es minimizar, la interfaz decide si el **SimplexSolver** requiere activar la bandera de inicialización con penalización M.

Resolucion de problemas lineales

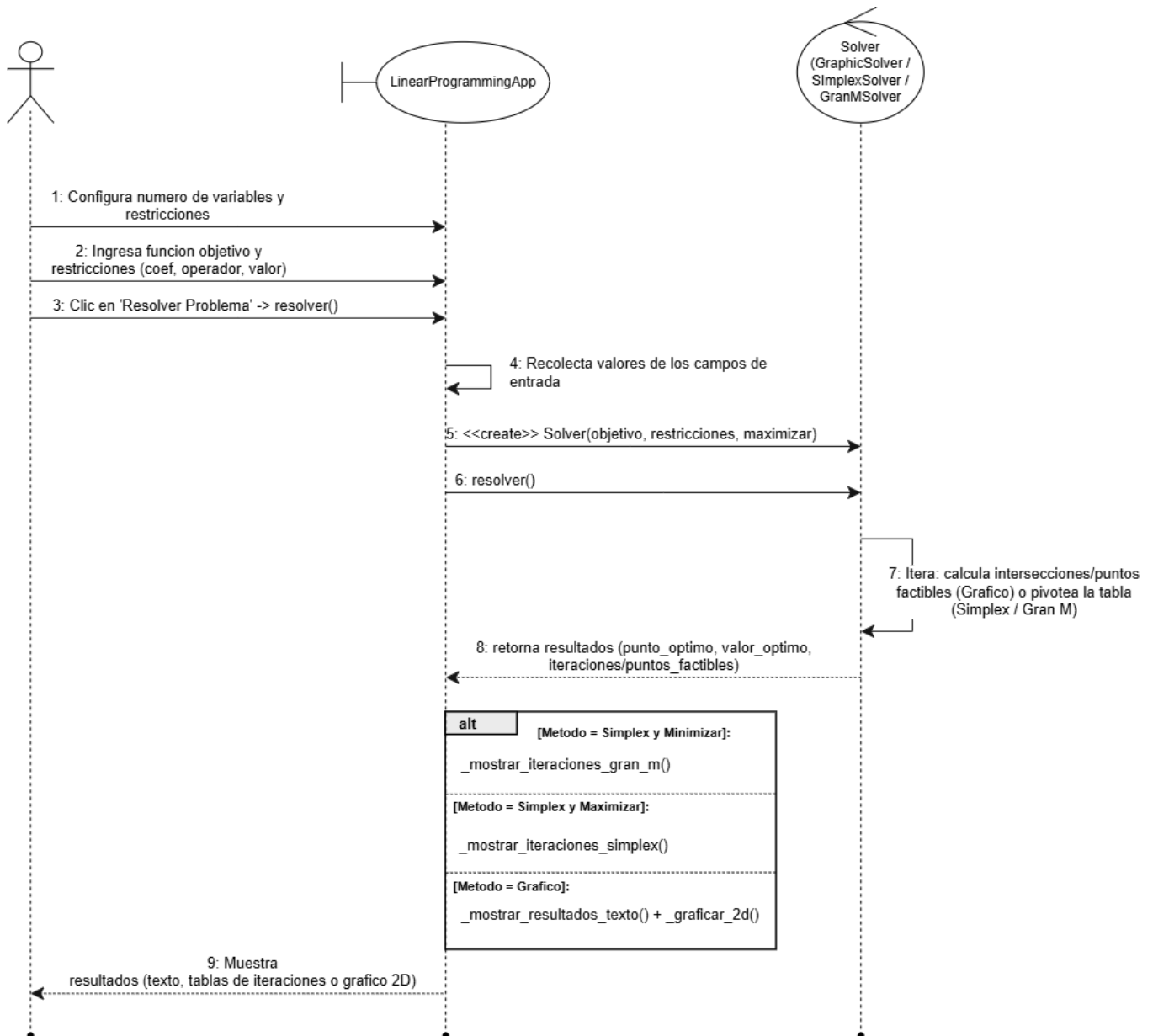


Figura 2. Diagrama de secuencia.

3.4 Implementación del método gráfico:

La clase `GraphicSolver` calcula los puntos de intersección entre cada par de restricciones (incluyendo los ejes) y conserva únicamente los que cumplen todas las condiciones del problema.

Resolucion de problemas lineales

```
1 for i in range(cant_restricciones):
2     for j in range(i + 1, cant_restricciones):
3         A = np.array([restricciones_completas[i]["coef"], restricciones_completas[j]["coef"]], dtype=float)
4         b = np.array([restricciones_completas[i]["val"], restricciones_completas[j]["val"]], dtype=float)
5
6         if np.linalg.matrix_rank(A) == 2:
7             pt = np.linalg.solve(A, b)
8             puntos.append(pt)
9
10    puntos_factibles = []
11    for pt in puntos:
12        x1, x2 = pt
13        if x1 < -1e-9 or x2 < -1e-9:
14            continue
15
16        valid = True
17        for r in restricciones_completas:
18            val = r["coef"][0] * x1 + r["coef"][1] * x2
19            op = r["op"]
20            target = r["val"]
21
22            if op == "<=" and val > target + 1e-9:
23                valid = False; break
24            elif op == ">=" and val < target - 1e-9:
25                valid = False; break
26            elif op == "=" and abs(val - target) > 1e-9:
27                valid = False; break
28
29        if valid:
30            puntos_factibles.append(pt)
```

Código 1. Cálculo de intersecciones y filtrado de puntos factibles.

Una vez obtenidos los vértices factibles, se evalúa la función objetivo en cada uno y se selecciona el máximo o el mínimo según corresponda, tal como en el ejemplo matemático mostrado anteriormente:

```
1 for p in puntos_factibles:
2     z = self.objetivo[0] * p[0] + self.objetivo[1] * p[1]
3     z_valores.append(z)
4
5     if self.maximizar:
6         indice_optimo = np.argmax(z_valores)
7     else:
8         indice_optimo = np.argmin(z_valores)
9
10    punto_optimo = puntos_factibles[indice_optimo]
11    valor_z_optimo = z_valores[indice_optimo]
```

Código 2. Evaluación en la función objetivo.

3.5 Implementación del Método Simplex:

Resolucion de problemas lineales

El núcleo del algoritmo simplex es un bucle que, en cada iteración, identifica la columna pivote (coeficiente más negativo de la fila objetivo), la fila pivote (criterio de razón mínima) y realiza el pivoteo sobre toda la tabla:

```
1 while True:
2     fila_z = tabla[-1, :-1]
3
4     if np.all(fila_z >= -1e-9):
5         break
6
7     col_pivote = np.argmin(fila_z)
8
9     col_valores = tabla[:-1, col_pivote]
10    lados_derechos = tabla[-1, -1]
11
12    cocientes = []
13    for i in range(num_restr):
14        if col_valores[i] > 1e-9:
15            cocientes.append(lados_derechos[i] / col_valores[i])
16        else:
17            cocientes.append(np.inf)
18
19    fila_pivote = np.argmin(cocientes)
20
21    if cocientes[fila_pivote] == np.inf:
22        return None
23
24    elemento_pivote = tabla[fila_pivote, col_pivote]
25
26    tabla[fila_pivote, :] /= elemento_pivote
27
28    for i in range(len(tabla)):
29        if i != fila_pivote:
30            factor = tabla[i, col_pivote]
31            tabla[i, :] -= factor * tabla[fila_pivote, :]
32
33    iteraciones.append(tabla.copy())
```

Código 3. Bucle principal del método Simplex

Este proceso se repite hasta que la fila objetivo no tenga coeficientes negativos, momento en que la tabla contiene la solución óptima. Finalmente, el sistema recorre las columnas de la tabla para identificar las variables básicas y extraer su valor:

```
1 for j in range(num_vars):
2     columna = tabla[:, j]
3     es_uno = np.isclose(columna, 1.0)
4     es_cero = np.isclose(columna, 0.0)
5
6     if np.count_nonzero(es_uno) == 1 and np.count_nonzero(es_cero) == len(columna) - 1:
7         fila_uno = np.where(es_uno)[0][0]
8         if fila_uno < num_restr:
9             solucion_vars[j] = tabla[fila_uno, -1]
10
11 valor_z_optimo = tabla[-1, -1]
```

Código 4. Interpretación de la tabla final

3.6 Implementación de la Gran M:

Resolucion de problemas lineales

Para implementar la Gran M dentro del solver algorítmico (exclusivo para minimización), se agregó una subrutina de preprocesamiento. Si el sistema detecta restricciones $\geq$, agrega automáticamente variables de holgura (negativas) y variables artificiales (positivas). Posteriormente, modifica la fila de la función objetivo sumando el valor M (definido computacionalmente como un número flotante arbitrariamente grande y positivo, e.g., $+1e6$) multiplicado por las variables artificiales.

```
1      slack_vars = []
2      artificial_vars = []
3
4      col_index = num_vars
5      var_info = []
6
7      for i, r in enumerate(self.restricciones):
8          if r["op"] == "<=":
9              slack_vars.append((col_index, i, 1.0))
10             var_info.append(("s", i))
11             col_index += 1
12          elif r["op"] == ">=":
13              slack_vars.append((col_index, i, -1.0))
14              var_info.append(("s", i))
15              col_index += 1
16
17              artificial_vars.append((col_index, i, 1.0))
18              var_info.append(("a", i))
19              col_index += 1
20          elif r["op"] == "=":
21              artificial_vars.append((col_index, i, 1.0))
22              var_info.append(("a", i))
23              col_index += 1
24
25      total_cols = col_index + 1
```

Código 5. Generación de variables artificiales y penalización M

Antes de iniciar las iteraciones estándar del Simplex, el algoritmo requiere una operación de "cerificación" (volver cero) de los coeficientes de las variables artificiales en la fila Z, utilizando las filas pivote de dichas restricciones.

Resolucion de problemas lineales

```
1  if np.all(fila_z >= -1e-9):
2      break
3
4      col_pivote = np.argmin(fila_z)
5
6      col_valores = tabla[:-1, col_pivote]
7      lados_derechos = tabla[:-1, -1]
8
9      cocientes = []
10     for i in range(num_restr):
11         if col_valores[i] > 1e-9:
12             cocientes.append(lados_derechos[i] / col_valores[i])
13         else:
14             cocientes.append(np.inf)
15
16     fila_pivote = np.argmin(cocientes)
17
18     if cocientes[fila_pivote] == np.inf:
19         return None
20
21     elemento_pivote = tabla[fila_pivote, col_pivote]
22     tabla[fila_pivote, :] /= elemento_pivote
23
24     for i in range(len(tabla)):
25         if i != fila_pivote:
26             factor = tabla[i, col_pivote]
27             tabla[i, :] -= factor * tabla[fila_pivote, :]
28
29     iteraciones.append(tabla.copy())
30
31     artificial_col_indices = [col_idx for col_idx, _, _ in artificial_vars]
32     for col_idx in artificial_col_indices:
33         columna = tabla[:, col_idx]
34         es_uno = np.isclose(columna, 1.0)
35         es_cero = np.isclose(columna, 0.0)
36         if np.count_nonzero(es_uno) == 1 and np.count_nonzero(es_cero) == len(columna) - 1:
37             fila_uno = np.where(es_uno)[0][0]
38             if fila_uno < num_restr and tabla[fila_uno, -1] > 1e-9:
39                 return None
40
```

Código 6. Operación de normalización de la fila Z inicial

3.7 Interfaz de usuario:

La interfaz se organiza en un panel izquierdo de configuración y un panel derecho de resultados, soportando ahora la renderización de variables artificiales (denotadas usualmente como A1, A2...) y mostrando el coeficiente M en las tablas iterativas.

Resolucion de problemas lineales

Configuración Inicial

Variables de decisión (n): 2
Cantidad restricciones: 2
Método: Simplex
Objetivo: Maximizar

Función Objetivo (Z)

x1: 1.0 x2: 1.0

Restricciones

1.0 x1 + 1.0 x2 <= 10.0
1.0 x1 + 1.0 x2 <= 10.0

Resultados y Tablas Simplex

Z Óptimo: 10.00 | Variables: x1 = 10.00, x2 = 10.00

--- Iteración 0 ---

x1	x2	s1	s2	CR
1.00	1.00	1.00	0.00	10.00
1.00	1.00	0.00	1.00	10.00
-1.00	-1.00	0.00	0.00	0.00

--- Iteración 1 ---

x1	x2	s1	s2	CR
1.00	1.00	1.00	0.00	10.00
0.00	0.00	-1.00	1.00	0.00
0.00	0.00	1.00	0.00	10.00

Figura 3. Interfaz gráfica principal del sistema

IV. RESULTADOS

El sistema fue probado con distintos problemas. En los casos de dos variables, el método gráfico identificó el punto óptimo ($Z = 36$ en el punto $(2,6)$). En los casos resueltos por simplex estándar (restricciones $\leq$), el sistema llegó a la solución óptima sin complicaciones. Para validar la implementación de la Gran M, se resolvieron casos de minimización con restricciones mixtas ($\leq$, $\geq$). El programa asignó correctamente la penalización enorme y positiva a las variables artificiales, forzándolas a salir de la base en las primeras iteraciones, alcanzando exitosamente la región factible real y posteriormente la solución óptima del problema original.

Resolucion de problemas lineales

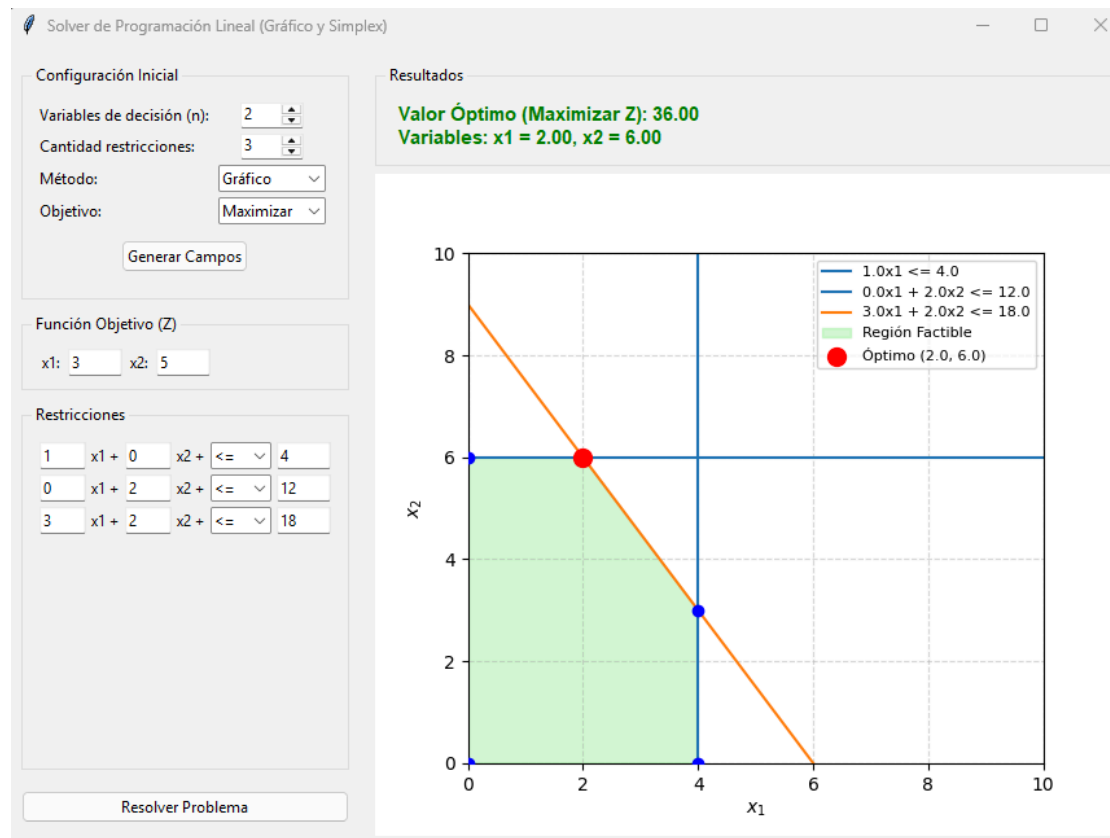
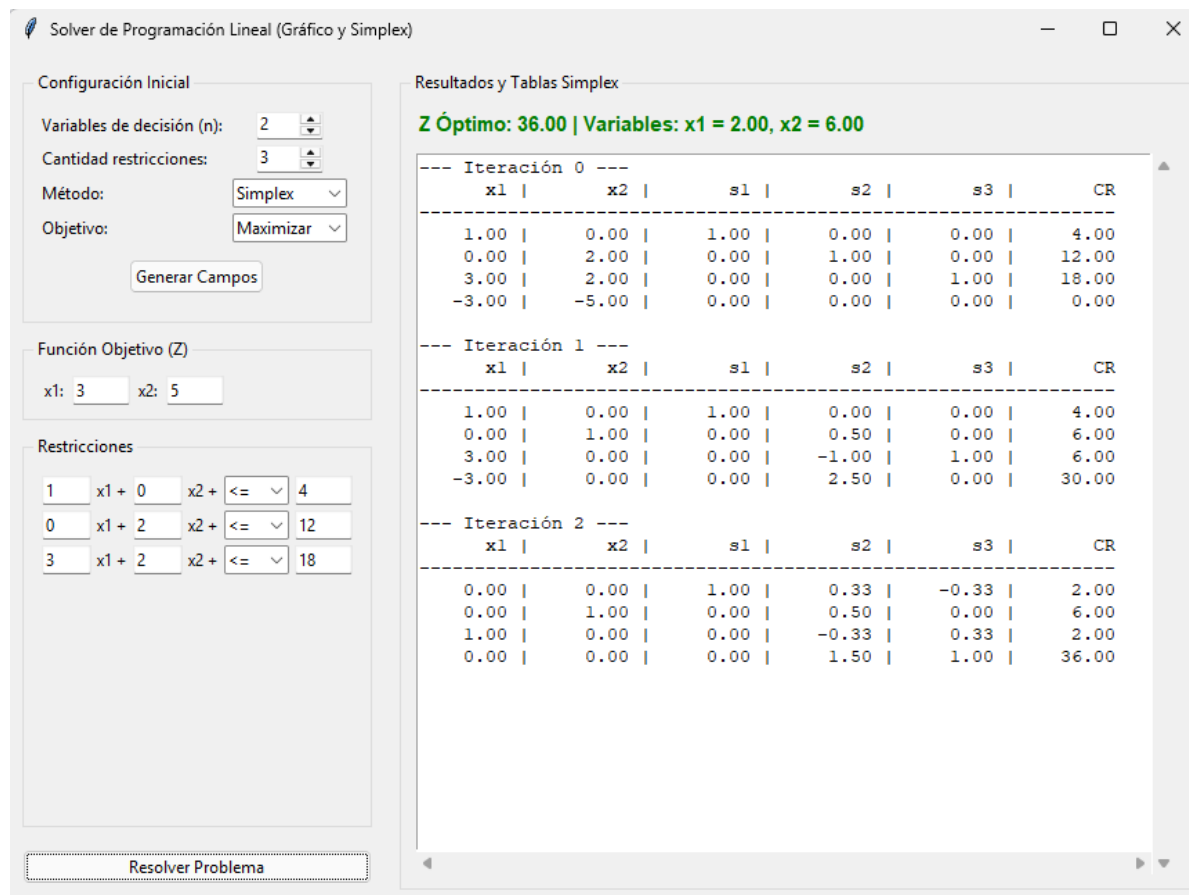


Figura 4. Resultado obtenidos para el ejemplo por método gráfico



Resolucion de problemas lineales

Figura 5. Resultado obtenidos para el ejemplo por método simplex

Para la validación visual y captura de pantalla correspondientes al Método de la Gran M, fue necesario emplear un modelo matemático distinto al ejemplo clásico formulado en el marco teórico. Dado que la Gran M se configuró de forma exclusiva para casos de minimización con restricciones mayor o igual, este nuevo ejemplo permitió evidenciar correctamente la generación de las variables artificiales y la aplicación de la penalización M en la tabla inicial del algoritmo.

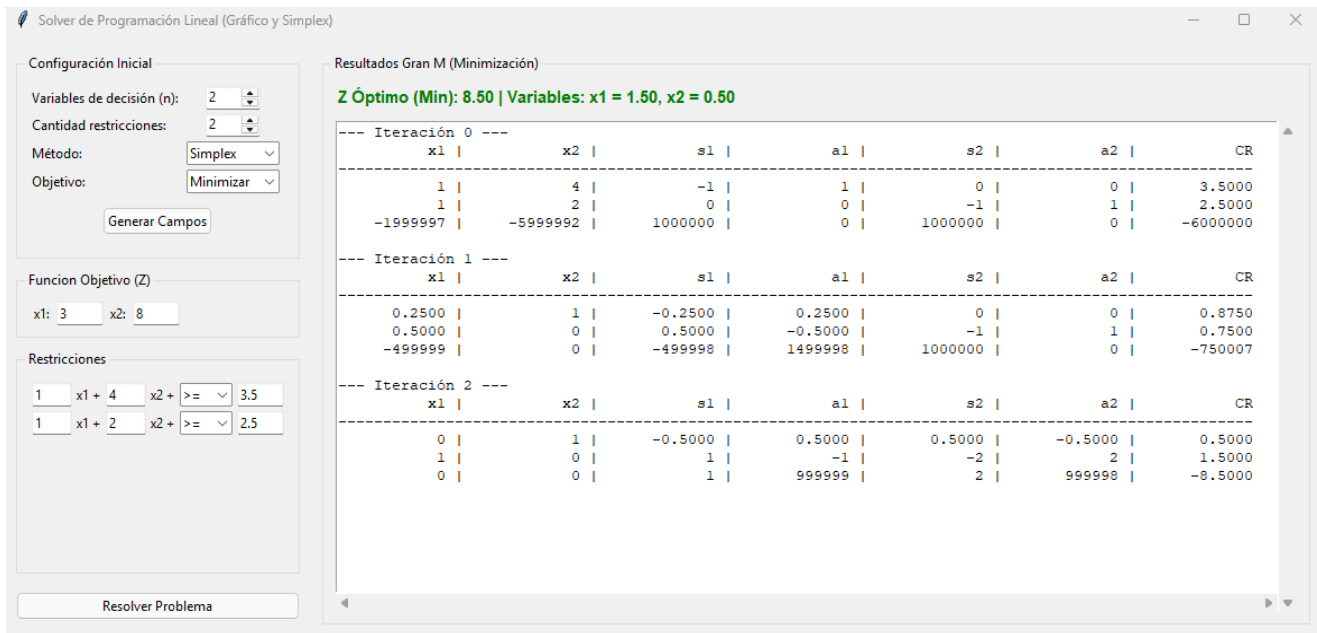


Figura 6. Resultado obtenidos para el ejemplo por método de la Gran M

V. CONCLUSIONES

El desarrollo del programa permitió comprobar de forma integral el funcionamiento del método gráfico, del método simplex y de su extensión mediante la Gran M orientada a la minimización. La inclusión de la Gran M demostró ser fundamental para dotar al software de robustez matemática, permitiéndole manejar modelos de programación lineal de minimización con cualquier combinación de restricciones. La arquitectura orientada a objetos facilitó la incorporación de las variables artificiales y penalizaciones dentro del mismo SimplexSolver, sin necesidad de reescribir la lógica de pivoteo. El uso de diagramas UML e historias de usuario resultó clave para adaptar los nuevos requerimientos. Como trabajo futuro, se proyecta la implementación del Método de las Dos Fases como alternativa algorítmica para evadir los problemas de desbordamiento numérico que el uso de un valor "M" excesivamente grande podría generar en máquinas con precisión limitada.

VI. REFERENCIAS

- [1] G. B. Dantzig, Linear Programming and Extensions. Princeton, NJ, USA: Princeton University Press, 1963.
- [2] H. A. Taha, Investigación de Operaciones, 9na ed. México: Pearson Educación, 2012.
- [3] Preparación de los documentos IEEE Transaction and Journals, diciembre, 2020.
- [4] F. S. Hillier y G. J. Lieberman, Introducción a la Investigación de Operaciones, 10ma ed. México: McGraw-Hill, 2015.